Preguntas de análisis

1. ¿Cómo reduce el Broker el acoplamiento?

El cliente no conoce ni instancia al proveedor; solo envía el nombre del servicio/operación al Broker, que se encarga de localizarlo y enrutar. Así se puede cambiar o sustituir el proveedor sin tocar al cliente.

2. ¿Qué detalles oculta al cliente?

La ubicación de red del proveedor, su implementación interna, base de datos, generación de QR y demás lógica de negocio. El cliente solo ve el contrato JSON de solicitud/respuesta.

3. ¿Qué sucede si el Broker deja de funcionar?

Se interrumpe la integración distribuida: las solicitudes no se enrutan y terminan en timeout/error. Cada sistema puede seguir funcionando localmente, pero sin comunicación entre ellos.

4. ¿El Broker constituye un punto único de falla?

Sí, en esta arquitectura simple es el punto único de falla para la comunicación entre cliente y proveedor.

5. ¿Cómo se podría mejorar su disponibilidad?

Con réplicas del Broker, failover, balanceo de carga, health checks, reconexión automática y reintentos limitados. También ayudan timeouts bien configurados y registro redundante.

6. ¿Qué atributo mejora al registrar servicios dinámicamente?

La flexibilidad (y con ella la mantenibilidad/escalabilidad): se pueden agregar o sustituir proveedores en caliente sin modificar al cliente.

7. ¿Qué costo introduce el Broker en rendimiento?

Un salto de red extra, serialización/deserialización JSON, enrutamiento y overhead de sockets/hilos. Puede volverse cuello de botella si no se dimensiona bien.

8. ¿Dónde se realiza el marshalling y unmarshalling?

En los tres componentes: el cliente serializa al enviar y parsea al recibir; el Broker convierte mensajes en tránsito; el proveedor hace lo propio. Se apoya en Gson en cada sistema.

9. ¿Cómo se gestionarían solicitudes duplicadas?

Con un id único por solicitud (idempotency key). El Broker/proveedor detectan repetidos por ese id, evitan reprocesar y pueden devolver la respuesta ya generada.

10. ¿Cómo se monitorearía el Broker?

Con logs por request id, métricas de latencia/throughput/conexiones/timeouts, health checks, alertas y monitoreo de recursos (CPU, memoria, hilos).